

实验 1：简易消息中间件开发

1 实验背景与目标

在分布式系统与微服务架构日益复杂的今天，消息中间件是实现系统解耦、异步通信和流量削峰的核心组件。本实验旨在通过“造轮子”的方式，深入理解事件驱动架构（Event-Driven Architecture, EDA）的底层原理。

核心目标如下：

掌握架构风格：深入理解事件系统型软件架构风格，区别于传统的调用/返回架构。

设计模式实战：熟练运用观察者模式或发布/订阅模式进行系统建模。

独立开发能力：在不依赖开源框架（如 RabbitMQ, Kafka, RocketMQ）的前提下，开发一个具备基础功能的单机版消息中间件。

非功能分析：具备对中间件吞吐率（Throughput）、延迟等性能指标的分析能力。

场景应用：能将中间件集成到实际的管理系统中，解决具体业务场景下的解耦问题。

2 任务详细说明

本实验要求以 2 结对形式完成，不要单人作战或三人组队。

● 开发约束（硬性要求）

禁止使用开源框架：不得使用现有的成熟消息中间件库。你可以使用基础的网络库（如 Java Socket, Netty 基础组件，或 Python 的 socket 库）和多线程库，但核心的“注册”、“发布”、“订阅”、“路由”逻辑必须手写。

架构强制：必须严格遵循事件驱动架构风格。

模式选择：必须在代码设计中明确体现 观察者/被观察者 或 发布/订阅 模式的结构。

● 功能需求（最低标准）

解耦能力：实现生产者（Publisher）与消费者（Subscriber）的完全解耦。

基本通信：支持消息的发送、接收与分发。

单机运行：至少保证在单机环境下，多个进程/线程间能够通过中间件进行通信。

场景模拟：需结合一个具体的“网络服务管理系统”（例如：校园卡管理系统、图书借阅系统、实验室预约系统等）来演示中间件的应用。

● 过程管理

结对编程：编码与测试阶段必须两人同时在场，共同完成。

影像记录：必须拍摄 结对编程的现场照片（建议包含屏幕代码特写和两人操作画面），作为过程考核的凭证。

文档驱动：遵循软件工程规范，每个阶段必须产出对应的文档和图表。

3 核心中间件技术

技术点	说明
网络通信	Socket / Netty，处理生产者与消费者的连接
线程模型	多线程处理并发请求，线程池管理
消息队列	阻塞队列（如 <code>LinkedBlockingQueue</code> ）管理待分发消息
订阅管理	存储主题与订阅者的映射关系
序列化	Java 序列化 / JSON，支持跨语言（可选）

4 服务端实现要点

要点	说明
多线程处理	每个客户端连接分配独立线程或使用线程池
线程安全	订阅关系存储使用 <code>ConcurrentHashMap</code>
消息队列	使用 <code>BlockingQueue</code> 实现异步缓冲
事件循环	独立线程消费队列并分发消息
心跳机制	检测客户端存活，清理失效订阅

5.6 客户端实现要点

要点	说明
连接管理	与服务端建立长连接（Socket）
消息封装	定义 Message 类，包含 topic、payload、timestamp
异步回调	接收服务端推送时触发 onEvent 回调
重连机制	断网时自动重连（可选增强）

5 实验交付物清单

请按照以下清单准备提交材料，确保文档与代码的一致性。

- （1）实验报告：完成实验报告模板中指定的内容。
- （2）程序。

6 考核标准与评分细则

本次实验成绩采用 “过程+结果” 的双重评价体系。

● 过程考核（60%）

结对证据（20%）：现场照片的真实性与数量，体现两人协作的过程。

文档质量（20%）：文档结构是否完整，建模图是否规范，分析是否深入。

设计合理性（20%）：架构风格运用是否得当，代码结构是否清晰体现了事件驱动思想。

● 程序结果（40%）

功能实现（20%）：程序能否正常运行，是否实现了观察者/被观察者模式或发布/订阅模式的基本功能。

性能分析（10%）：是否进行了吞吐率等指标的量化分析，分析方法是否科学。

应用价值（10%）：结合的实际系统场景是否合理，中间件是否真正起到了解耦作用。